

ASSIGNMENT-8

NAME: SRAVANI ANUMALA

ROLLNO: 2303A510G7

BATCH: 30

TASK:

An Overview of Security Pitfalls in Smart Contract

Development

Objective:

To understand and demonstrate common smart contract security vulnerabilities by implementing an insecure smart contract in Solidity, identifying its weaknesses, and fixing them using best security practices.

Problem Statement:

Smart contracts are immutable and handle real value, making them attractive targets for attacks. This practical demonstrates common security pitfalls such as Reentrancy, Integer Overflow/Underflow, and Improper Access Control, and shows how to mitigate them.

Requirements:

- Install VS Code
- Install Solidity extension in VS Code
- Use Remix IDE (online) or Hardhat for testing
- Basic knowledge of Ethereum & Solidity
- MetaMask (optional, for deployment testing)

CODE:

```
File Edit Selection View Go Run Terminal Help
assignment-6.py Block Chain-ASSIGNMENT-7-1007.pdf ass-8.html x
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Blockchain Assignment - Insecure Smart Contract</title>
7 <style>
8   body {
9     font-family: Arial, sans-serif;
10    line-height: 1.6;
11    background-color: #f4f4f4;
12    padding: 20px;
13    color: #333;
14  }
15  h1, h2, h3 {
16    color: #2c3e50;
17  }
18  pre {
19    background-color: #f0f0f0;
20    padding: 10px;
21    overflow-x: auto;
22  }
23  code {
24    font-family: monospace;
25  }
26  .section {
27    background-color: #fff;
28    margin-bottom: 20px;
29    padding: 20px;
30    border-radius: 8px;
31    box-shadow: 0 0 8px rgba(0,0,0,0.1);
32  }
33  .highlight {
34    color: #e3302b;
35    font-weight: bold;
36  }
37 </style>
38 </head>
39 <body>
40
41 <div class="section">
42 <h1>Blockchain Assignment: Security Pitfalls in Smart Contracts</h1>
43 <p><strong>Course:</strong> Blockchain ()</p>
44 <p><strong>Submitted by:</strong> [Sravani Anumala]</p>
45 <p><strong>Academic Year:</strong> 2023-2027</p>
46 </div>
47
48 <div class="section">
```

Blockchain Assignment - Insecure... x

127.0.0.1:5500/ass-8.html

Summarize Sign in Update Chat

Blockchain Assignment: Security Pitfalls in Smart Contracts

Course: Blockchain ()
Submitted by: [Sravani Anumala]
Academic Year: 2023-2027

1. Objective

Understand and demonstrate common smart contract security vulnerabilities by implementing an insecure smart contract, identifying weaknesses, and fixing them using best security practices.

2. Problem Statement

Smart contracts handle real value and are immutable, making them vulnerable to attacks. This assignment demonstrates:

- Reentrancy attacks
- Integer Overflow / Underflow
- Improper Access Control

3. Solidity Smart Contract (InsecureBank.sol)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract InsecureBank {
    mapping(address => uint) public balances;
```

Blockchain Assignment - Insecure: X

127.0.0.1:5500/ass-8.html

Summarize Sign in Update Chat

3. Solidity Smart Contract (InsecureBank.sol)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.8;

contract InsecureBank {
    mapping(address => uint) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint _amount) public {
        require(balances[msg.sender] >= _amount, "Insufficient balance");
        payable(msg.sender).call{value: _amount}(""); // Vulnerable
        balances[msg.sender] -= _amount;
    }

    function getBalance() public view returns (uint) {
        return balances[msg.sender];
    }
}
```

Vulnerabilities:

- Reentrancy – Withdraw sends Ether before updating balance
- Access Control – Anyone can call withdraw for their own balance

4. Security Fix

```
function withdraw(uint _amount) public {
    require(balances[msg.sender] >= _amount, "Insufficient balance");

    balances[msg.sender] -= _amount; // Update balance first
    payable(msg.sender).transfer(_amount); // Safe transfer
}
```

Fix applied: Reentrancy prevented by updating balance before sending Ether.

Blockchain Assignment - Insecure: X

127.0.0.1:5500/ass-8.html

Summarize Sign in Update Chat

4. Security Fix

```
function withdraw(uint _amount) public {
    require(balances[msg.sender] >= _amount, "Insufficient balance");

    balances[msg.sender] -= _amount; // Update balance first
    payable(msg.sender).transfer(_amount); // Safe transfer
}
```

Fix applied: Reentrancy prevented by updating balance before sending Ether.

5. Deployment Steps (Conceptual)

1. Install Hardhat in your project folder: `npm install --save-dev hardhat`
2. Create Hardhat project: `npm run hardhat`
3. Compile the contract: `npm run hardhat compile`
4. Deploy using deploy.js script
5. Test deposit and withdraw functions

6. Observations

- Smart contracts must handle funds securely.
- Reentrancy is a critical vulnerability.
- Proper order of state updates is essential.
- Modern Solidity versions prevent integer overflow/underflow.

7. References

Blockchain Assignment - Insecure

127.0.0.1:5500/ass-8.html

Summarize

Sign in

Update

Chat

```
payable(msg.sender).transfer(amount); // Safe transfer
}
```

Fix applied: Reentrancy prevented by updating balance before sending Ether.

5. Deployment Steps (Conceptual)

1. Install Hardhat in your project folder: `npm install --save-dev hardhat`
2. Create Hardhat project: `npm hardhat`
3. Compile the contract: `npm hardhat compile`
4. Deploy using deploy.js script
5. Test deposit and withdraw functions

6. Observations

- Smart contracts must handle funds securely.
- Reentrancy is a critical vulnerability
- Proper order of state updates is essential.
- Modern Solidity versions prevent integer overflow/underflow.

7. References

- [Solidity Official Documentation](#)
- [Hardhat Documentation](#)
- [Ethereum Smart Contract Best Practices](#)

Blockchain Assignment - Insecure

Solidity -- Solidity 0.8.35 dev

https://docs.soliditylang.org/en/v0.8.35-pre-1/

Summarize

Sign in

Update

Chat

Search docs

BASICS

Introduction to Smart Contracts

Solidity by Example

Installing the Solidity Compiler

LANGUAGE DESCRIPTION

Layout of a Solidity Source File

Structure of a Contract

Types

Units and Globally Available Variables

Expressions and Control Structures

Contracts

Inline Assembly

Cheatsheet

Language Grammar

COMPILER

Using the Compiler

Analysing the Compiler Output

Documentation / Solidity

Edit on GitHub

Solidity

Solidity is an object-oriented, high-level language for implementing smart contracts. Smart contracts are programs that govern the behavior of accounts within the Ethereum state.

Solidity is a [curly-bracket language](#) designed to target the Ethereum Virtual Machine (EVM). It is influenced by C++, Python, and JavaScript. You can find more details about which languages Solidity has been inspired by in the [language influences](#) section.

Solidity is statically typed, supports inheritance, libraries, and complex user-defined types, among other features.

With Solidity, you can create contracts for uses such as voting, crowdfunding, blind auctions, and multi-signature wallets.

When deploying contracts, you should use the latest released version of Solidity. Apart from exceptional cases, only the latest version receives [security fixes](#). Furthermore, breaking changes, as well as new features, are introduced regularly. We currently use a 0.y.z version number to [indicate this fast pace of change](#).

Warning

Solidity recently released the 0.8.x version that introduced a lot of breaking changes. Make sure you read [the full list](#).

Ideas for improving Solidity or this documentation are always welcome, read our [contributors guide](#) for more details.

en

v0.8.35-pre-1

